Bias-Aware Indic Spoken Language Identification

Project Team
Saarland University
spoken-language-identification

Abstract

This report studies Indic spoken language identification on badrex/nnti-dataset-full. We reproduce the baseline direction and improve a strong mHuBERT reference through tuning (0.4676 → 0.5270 accuracy). We then evaluate multiple speaker-bias mitigation variants and find that model-based adversarial training (DANN) gives the best final result: 0.5576 accuracy and 0.5426 macro-F1. We analyze bias source, mitigation success, confusion patterns, and last-layer t-SNE behavior. Results show clear bias reduction and weak-class recovery, but not complete removal of speaker information.

1 Introduction

This project studies spoken language identification under strong speaker-shift conditions. The key challenge is that the model must learn language cues from a limited set of training speakers and generalize to fully unseen validation speakers.

Implementation uses PyTorch (Paszke et al., 2019), torchaudio augmentation practice from the PyTorch Audio tutorial, Hugging Face Transformers/Datasets (Wolf et al., 2020), scikit-learn (Pedregosa et al., 2011), Matplotlib (Hunter, 2007), MMS (Pratap et al., 2023), wav2vec2 (Baevski et al., 2020), HuBERT (Hsu et al., 2021), DANN (Ganin et al., 2016), and t-SNE (van der Maaten and Hinton, 2008).

2 Dataset

We use badrex/nnti-dataset-full (Badr et al., 2026). Core split statistics:

- Languages: 22
- Train samples: 8689
- Validation samples: 3300
- Train unique speakers: 110
- Validation unique speakers: 681
- Train/validation speaker overlap: 0

This is a strict unseen-speaker setting because all validation samples are from speakers absent in training.

3 experiments

3.1 experimental setup (over all experientmental settings)

To keep comparisons fair, we kept data split and seed fixed and changed only intended components per run. Shared protocol:

- Fixed split: same train/validation files across runs.
- Fixed seed: 42.
- Best-checkpoint loading enabled (load_best_model_at_end=True).
- Same diagnostics family: per-language accuracy, confusion matrix, speaker diagnostics.

3.2 eval metrics (overall eval metrics used)

We report:

- Global metrics: validation accuracy, macro-F1, eval loss.
- Class-level metric: per-language validation accuracy.
- Error structure: off-diagonal confusion pairs from confusion matrix.
- Speaker diagnostics: overlap count and unseen-speaker accuracy.
- Representation diagnostics: last-layer t-SNE (where artifacts are available).

For final ranking in speaker-bias mitigation, macro-F1 is prioritized with accuracy as a guardrail.

4 Baseline system and improvements

4.1 Methods

The baseline-improvement pipeline followed two stages:

- **Baseline direction reproduction**: supervised fine-tuning from assignment baseline family (MMS-300m setup).
- **Improvement stage**: backbone and hyperparameter improvements leading to tuned mHuBERT reference.

In the improvement stage, we focused on reproducible tuning rather than architecture redesign: learning-rate/schedule tuning, checkpoint-selection strategy, and stable training controls. Reporting these hyperparameter-level changes is useful because final quality depends strongly on training setup, not only on backbone choice.

The main hyperparameter settings reported for this baseline-improvement stage are: target learning rate (around 1×10^{-5} with warmup schedule), effective batch control (per-device batch size with gradient accumulation), checkpointing policy (`load_best_model_at_end=True`), model-selection metric (accuracy for the tuned no-mitigation reference), and evaluation/save cadence during training. These settings are part of the method because they materially changed final quality.

The tuned mHuBERT reference became the no-mitigation control for speaker-bias mitigation experiments.

4.2 experiments

We executed three experimental checkpoints in sequence: (i) baseline direction using MMS-300m reference performance from the assignment, (ii) untuned mHuBERT run, and (iii) tuned mHuBERT run. This sequence isolates the effect of model choice and tuning quality before any mitigation logic is added.

Run	Acc	Macro-F1	Eval Loss
Assignment baseline (MMS-300m, provided)	0.3196	–	–
mHuBERT untuned (`20260301_094826`)	0.4676	0.4142	2.2854
mHuBERT tuned (`20260301_150502`)	**0.5270**	**0.4748**	**2.1629**

Table 1: Baseline and improvement experiment summary.

4.3 results

In baseline-direction reproduction, the provided MMS-300m baseline is 31.96% top-1 accuracy. The untuned mHuBERT run already exceeds that level (0.4676), and tuned mHuBERT further improves to 0.5270 accuracy and 0.4748 macro-F1. Relative to untuned mHuBERT, tuning adds +0.0594 accuracy and +0.0607 macro-F1. This tuned run is the direct control for all mitigation comparisons.

5 Speaker-Bias Mitigation

5.1 Methods

We used both data-centric and model-based mitigation.

Data-centric mitigation (train-only):

- Mitigation 1: speed/noise augmentation ($p = 0.30$, speed 0.97–1.03, noise std 0.0005–0.002).
- Mitigation 2: gentler speed/noise ($p = 0.25$, speed 0.98–1.02, noise std 0.0005–0.0015) with macro-F1 checkpointing.
- Mitigation 3A: revert to M1 augmentation strength, macro-F1 checkpointing, denser eval/save cadence (100/100).
- Mitigation 3B: milder fallback from 3A ($p = 0.28$, speed 0.975–1.025, noise std max 0.0018).
- Mitigation 4: all data-centric mix (speed/noise + pitch shift + spectral masking) with global augmentation gate and t-SNE export.

Model-based mitigation (DANN):

- Shared encoder + language head + adversarial speaker head.
- Gradient reversal layer to suppress speaker-identifiable directions.
- Final DANN settings: `speaker_loss_weight=0.1`, `grl_lambda=1.0`, lambda schedule enabled, speaker-head dropout 0.1.
- Final DANN run intentionally used augmentation off to isolate model-based effect.

Encoder objective:

$$\mathcal{L}_{enc} = \mathcal{L}_{lang} - \lambda \mathcal{L}_{speaker}.$$

5.2 experiments

5.3 results

Key outcomes:

- DANN outperforms tuned no-mitigation by +0.0306 accuracy and +0.0678 macro-F1.
- DANN outperforms best data-centric run (Mitigation 1) by +0.0188 accuracy and +0.0558 macro-F1.

Run	Acc	Macro-F1	Eval Loss
Tuned ref (no mitigation)	0.5270	0.4748	2.1629
Mitigation 1	0.5388	0.4868	2.1504
Mitigation 2	0.5279	0.4828	2.1385
Mitigation 3A	0.4979	0.4561	2.1884
Mitigation 3B	0.5276	0.4817	2.1456
Mitigation 4	0.5115	0.4664	2.2127
DANN (final improved)	**0.5576**	**0.5426**	**1.8815**

Table 2: Mitigation experiments (validation split).

- Data-centric methods are sensitive to augmentation strength: Mitigation 1 worked best; stronger mixed augmentation (Mitigation 4) underperformed.

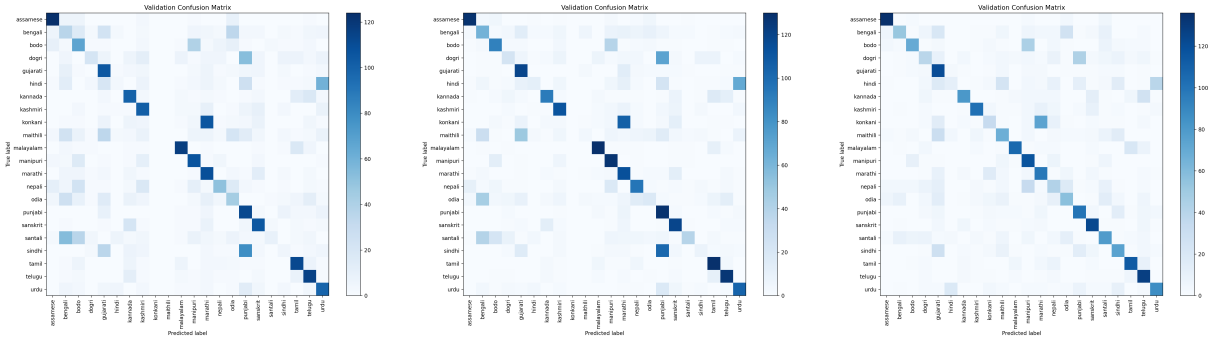

Figure 1: Validation confusion matrices: untuned reference (left), Mitigation 1 (middle), DANN improved (right).

6 analysis

6.1 Discuss the source of the bias in the data and explain why learning from biased data is difficult

The bias source is limited speaker diversity in training (110 speakers over 22 languages) plus fully unseen validation speakers (681). This creates a shortcut risk: the model can use speaker timbre/prosody as an easier proxy for language. Learning becomes difficult because weak classes get under-separated while related languages share acoustic patterns.

Evidence from this project:

- Untuned reference has near-zero classes (for example maithili 0.0000, konkani 0.0000, sindhi 0.0333).
- Baseline confusion mass concentrates in a few pairs (for example konkani→marathi and sindhi→punjabi).

6.2 Discuss your proposed technique and how it addresses the speaker bias problem. To what extent was the solution successful?

Our approach combines controlled data-centric variability and model-based adversarial de-biasing.

- Data-centric mitigation (especially Mitigation 1) improves robustness by breaking fixed speaker-style dependence.
- DANN directly penalizes speaker-identifiable features in the shared encoder.

Success extent is strong but partial:

- Unseen-speaker accuracy improves from 0.4676 (untuned) to 0.5576 (DANN).
- Weak-class recovery is substantial (for example sindhi 0.0333 → 0.5000, maithili 0.0000 → 0.4467).
- Some previously strong classes regress, so bias reduction is not complete.

Pair (true→pred)	Untuned	DANN	Δ
konkani→marathi	106	74	-32
sindhi→punjabi	79	14	-65
hindi→urdu	60	39	-21
santali→bengali	54	10	-44
dogri→punjabi	53	44	-9
bodo→manipuri	39	45	+6

Table 3: Top confusion shifts from untuned reference to DANN.

6.3 Analyze the confusion patterns between languages. Which languages are most often confused with each other? why?

Most frequent confusions are:

- `konkani→marathi`
- `sindhi→punjabi`
- `hindi→urdu`

These are plausible due to linguistic/acoustic proximity and insufficient class-separating cues in short utterances. DANN reduces several high-volume confusions, but not all.

6.4 Analyze the baseline and the improved model by visualization the representations of the last layer using the TSNE algorithm. Discuss whether or not the models encode the speaker identity

t-SNE is applied to last-layer validation embeddings. In local final artifacts, full t-SNE exports are available for DANN; untuned/tuned/Mitigation-1 t-SNE exports are not fully available in this branch snapshot.

For DANN:

- Silhouette (language): 0.0047
- Silhouette (speaker top-k): -0.0992
- kNN speaker purity@5: 0.3433

Interpretation: speaker structure is reduced but still present, so speaker identity is not fully re-

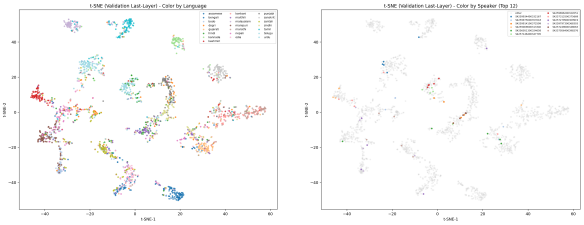

Figure 2: DANN t-SNE on validation embeddings: by language (left), by speaker (right).

Run	WeakMean	Top5Mass	BestStepRatio	UnseenAcc
Untuned ref	0.0227	0.2003	0.9511	0.4676
Tuned ref	0.0573	0.2511	0.9204	0.5270
Mitigation 1	0.0840	0.2602	0.8897	0.5388
DANN	**0.3547**	**0.1610**	0.9971	**0.5576**

Table 4: Additional analysis summary from run artifacts.

moved from representations.

6.5 Any other analyses you find insightful for your model

This shows DANN improves weak-class behavior and reduces concentration of errors into a few dominant confusion pairs.

7 conclusion

Tuned mHuBERT is a strong no-mitigation reference. Mitigation quality is configuration-sensitive: mild data-centric augmentation helps, while overly aggressive augmentation can hurt. The best final model is DANN, which gives the strongest overall performance and better weak-class recovery under strict unseen-speaker validation. Analysis confirms meaningful bias reduction, with remaining hard confusion pairs and residual speaker signal.

Limitations

- Full baseline-vs-improved t-SNE comparison is limited by missing historical t-SNE exports for some earlier runs in the current local artifact set.
- Some language pairs remain structurally difficult (for example konkani/marathi and hindi/urdu).
- AI usage disclosure: this report text was drafted with OpenAI Codex assistance; all run IDs, metrics, and claims were verified against local artifact files before inclusion.

References

Badr et al. 2026. Nnti dataset full. https://huggingface.co/datasets/badrex/nnti-dataset-full. Hugging Face dataset card, accessed 2026-03-05.

Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. 2020. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems*.

Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. 2016. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35.

Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. 2021. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460.

John D. Hunter. 2007. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.

Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Souvik Kundu, Ali Elkahky, Zhaoheng Ni, Apoorv Vyas, Ankur Bapna, et al. 2023. Scaling speech technology to 1,000+ languages. *arXiv preprint arXiv:2305.13516*.

PyTorch Audio Team. 2026. Audio data augmentation tutorial. https://docs.pytorch.org/audio/stable/tutorials/audio_data_augmentation_tutorial.html. PyTorch Audio documentation, accessed 2026-03-06.

Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(86):2579–2605.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz,

and Jamie Brew. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.